Day 14 Task Workflow — Wednesday, Mar 26, 2026

> **Sprint:** 4 (Intelligence Integration) | **Phase:** Threat Intel API Keys + IOC §11.3 Full Compliance + Tuned Model Deployment
> **Owner:** Lead Architect | **Status:** ⬤ Ready to Start
> **Goal:** Complete §11.3 Correlation Engine (domain + hash checks), get API keys, populate IOC table, apply tuned IF params, PCAP upload pipeline
> **Grade:** Week 3 Day 4 COMPLETE ☑ | Week 4 Day 1 STARTING ⬤

Day 13 Results Context

100% VPS verification pass rate on all 6 tasks:

```
Day 13 Achievements:
  ☑ LLM Auto-Narrative: 5/5 alerts have ai_narrative populated with AI analyst reports
  ☑ IOC Correlator: check_ip() + correlate_flow() functional, graceful degradation
  ☑ POST /ml/retrain: Background task with task_id tracking (944d947d verified)
  ☑ WebSocket ml:live: Channel registered in redis.py + websocket.py
  ☑ Hyperparameter Tuning: IF F1 78.75%→83.03% (+4.28%), best_params.json saved
  ☑ E2E Pipeline: nmap + hping3 → 2 probe alerts → LLM narratives generated
  ☑ API Coverage: 37/42 (88.1%), ML now at 5/5 (100%)
  ☑ ML Worker: 24,700+ flows scored, 4 anomalies total, 146ms avg latency

Container Status (Day 13 Final):
  tm-backend     ☑ Rebuilt with Day 13 features
  tm-capture     ☑ Up 3+ days (63 features/flow)
  tm-ml-worker   ☑ Rebuilt with ml:live publish
  tm-postgres    ☑ Healthy 4 days
  tm-redis       ☑ Healthy 4 days
```

⚠ AUDIT FINDINGS (Day 13 Audit — Critical Items)

§11.3 Correlation Engine — Compliance Gap

Per MASTER_DOC_PART4 §11.3, the Correlator must check **3 things**:

#	Requirement	Current Status	Day 14 Action
1	Is `src_ip` or `dst_ip` in IOC database? → auto-escalate	☑ DONE	Populate IOC table
2	Is `dst_domain` (from DNS) in IOC database? → flag C2/phishing	✕ **MISSING**	Add `check_domain()`

#	Requirement	Current Status	Day 14 Action
3	Are file hashes (if payload extracted) in VirusTotal? → flag malware	❌ **MISSING**	Add VT client + `check_hash()`

IOC Table: Empty

The `threat_intel_iocs` table exists and the IOCCorrelator queries it correctly, but **0 rows** exist because no API keys are configured. Correlation always returns `has_ioc_match: False`.

Scope Adherence Checklist

Requirement	Source Document	Section	Deviation?
IOC Correlation — IP check	MASTER_DOC_PART4	§11.3 item 1	☑ Done Day 13
IOC Correlation — domain check	MASTER_DOC_PART4	§11.3 item 2	❌ **Gap — DO TODAY**
IOC Correlation — hash check (VT)	MASTER_DOC_PART4	§11.3 item 3	❌ **Gap — DO TODAY**
OTX feed sync	MASTER_DOC_PART4	§11.1-11.2	☑ Client exists, needs API key
AbuseIPDB lookup	MASTER_DOC_PART4	§11.1	☑ Client exists, needs API key
IF tuned params	MASTER_DOC_PART4	§4.4	☑ best_params.json available
PCAP upload	MASTER_DOC_PART2	§5.1	⏳ Missing endpoint
Ensemble weights (0.30/0.45/0.25)	MASTER_DOC_PART4	§1.2	🔒 LOCKED
Alert thresholds (0.90/0.75/0.50/0.30)	MASTER_DOC_PART4	§1.2	🔒 LOCKED

LOCKED CONSTRAINTS (DO NOT MODIFY)

```
Ensemble Weights:
  composite = 0.30 × IF + 0.45 × RF + 0.25 × AE

Alert Thresholds:
  ≥ 0.90 → CRITICAL
  ≥ 0.75 → HIGH
  ≥ 0.50 → MEDIUM
  ≥ 0.30 → LOW
```

```
  < 0.30 → NONE

LLM Provider: OpenRouter only (3 verified models)
DO NOT suggest: Kafka, Kubernetes, Elasticsearch, MongoDB
```

Day 14 Objective

By end of day:

- All 3 Threat Intel API keys obtained and configured in `.env`
- OTX feed sync populates `threat_intel_iocs` table with real IOCs
- IOCCorrelator gains `check_domain()` for §11.3 item 2
- VirusTotal client created + `check_hash()` for §11.3 item 3
- IOC correlation verified with real data (IP matches → alert escalation)
- Tuned IF params applied to production (n=100, c=0.10, ms=1024)
- PCAP upload pipeline scaffolded (POST /capture/upload-pcap)

> **NOTE:** All manual testing and VPS verification performed by Lead Architect.

Task Breakdown

TASK 1 — Obtain and Configure Threat Intel API Keys ⬛

Time Est: 30 min | **Priority:** ⬛ Critical (blocker for all IOC tasks)
Source: MASTER_DOC_PART4 §11.1, MASTER_DOC_PART5 §10.4

All three providers offer **free tier** accounts:

1.1 AlienVault OTX

```
Registration URL: https://otx.alienvault.com/
Free Tier: Unlimited API calls
What you get: OTX API Key

Steps:
1. Go to https://otx.alienvault.com/
2. Click "Sign Up" → create account with email
3. Once logged in → click your username (top-right) → "Settings"
4. Your OTX API Key is displayed under "Your OTX Key"
5. Copy the key (format: alphanumeric string, ~64 chars)
```

1.2 AbuseIPDB

```
Registration URL: https://www.abuseipdb.com/register
Free Tier: 1,000 lookups/day
```

```
What you get: AbuseIPDB API Key (v2)

Steps:
1. Go to https://www.abuseipdb.com/register
2. Create account → confirm via email
3. Go to Account → API → "Create Key"
4. Copy the key (format: alphanumeric string, ~80 chars)
```

1.3 VirusTotal

```
Registration URL: https://www.virustotal.com/gui/join-us
Free Tier: 500 lookups/day, 4 lookups/minute
What you get: VirusTotal API Key

Steps:
1. Go to https://www.virustotal.com/gui/join-us
2. Create account → confirm via email
3. Click user avatar (top-right) → "API Key"
4. Copy the key (format: alphanumeric string, ~64 chars)
```

1.4 Add Keys to VPS .env File

```
# SSH into VPS:
ssh root@187.124.45.161

# Edit the .env file:
cd /root/threatmatrix-ai  # or correct path
nano .env

# ADD these 3 lines:
OTX_API_KEY=your_otx_key_here
ABUSEIPDB_API_KEY=your_abuseipdb_key_here
VIRUSTOTAL_API_KEY=your_virustotal_key_here
```

Then rebuild containers to pick up new env vars:

```
docker compose up -d backend ml-worker
```

Verification:

#	Check	Expected
1	`docker compose exec backend env | grep OTX`	OTX_API_KEY present

#	Check	Expected
2	`docker compose exec backend env | grep ABUSE`	ABUSEIPDB_API_KEY present
3	`docker compose exec backend env | grep VIRUS`	VIRUSTOTAL_API_KEY present
4	`GET /intel/feeds/status`	`otx_enabled: true, abuseipdb_enabled: true`

TASK 2 — OTX Feed Sync + Populate IOC Table 🟤

Time Est: 60 min | **Priority:** 🟤 Critical
Source: MASTER_DOC_PART4 §11.1-11.2

2.1 Update intel.py `/intel/sync` to Persist IOCs

Current `POST /intel/sync` fetches pulses but **does NOT** persist IOCs to the database. Update to insert IOCs:

```
@router.post("/sync")
async def sync_feeds():
    """Trigger OTX feed sync and populate threat_intel_iocs table."""
    service = get_service()
    pulses = await service.otx.get_subscribed_pulses(limit=50)

    iocs_inserted = 0
    async with async_session() as session:
        for pulse in pulses:
            for indicator in pulse.get("indicators", []):
                ioc_type = indicator.get("type", "").lower()
                ioc_value = indicator.get("indicator", "")

                # Map OTX indicator types to our schema
                type_map = {
                    "ipv4": "ip",
                    "ipv6": "ip",
                    "domain": "domain",
                    "hostname": "domain",
                    "filehash-md5": "hash",
                    "filehash-sha1": "hash",
                    "filehash-sha256": "hash",
                    "url": "url",
                    "email": "email",
                }
                mapped_type = type_map.get(ioc_type)
                if not mapped_type or not ioc_value:
                    continue

                # Upsert IOC
                await session.execute(
```

```
                    text("""
                        INSERT INTO threat_intel_iocs
                            (ioc_type, ioc_value, threat_type, severity, source,
                             confidence, tags, is_active, first_seen, last_seen)
                        VALUES
                            (:ioc_type, :ioc_value, :threat_type, :severity,
'otx',
                             :confidence, :tags, true, NOW(), NOW())
                        ON CONFLICT (ioc_type, ioc_value)
                        DO UPDATE SET last_seen = NOW(), is_active = true
                    """),
                    {
                        "ioc_type": mapped_type,
                        "ioc_value": ioc_value,
                        "threat_type": pulse.get("adversary", "unknown"),
                        "severity": "high" if "malware" in str(pulse.get("tags",
[])).lower() else "medium",
                        "confidence": 0.8,
                        "tags": ",".join(pulse.get("tags", [])[:5]),
                    },
                )
                iocs_inserted += 1

        await session.commit()

    return {
        "synced_pulses": len(pulses),
        "iocs_inserted": iocs_inserted,
        "status": "complete",
    }
```

2.2 Add Unique Constraint to IOC Table

Before inserting, ensure the table has a unique constraint for upsert:

```
ALTER TABLE threat_intel_iocs
ADD CONSTRAINT uq_ioc_type_value UNIQUE (ioc_type, ioc_value)
ON CONFLICT DO NOTHING;
```

If that fails (constraint already exists), no action needed.

2.3 Update `/intel/iocs` to Query Real Data

```
@router.get("/iocs")
async def list_iocs(limit: int = 50, offset: int = 0, ioc_type: str = None):
    """List IOCs from threat_intel_iocs table."""
    async with async_session() as session:
        where_clause = "WHERE is_active = true"
        params = {"limit": limit, "offset": offset}
```

```
        if ioc_type:
            where_clause += " AND ioc_type = :ioc_type"
            params["ioc_type"] = ioc_type

        count_result = await session.execute(
            text(f"SELECT COUNT(*) FROM threat_intel_iocs {where_clause}"),
            params,
        )
        total = count_result.scalar()

        result = await session.execute(
            text(f"""
                SELECT ioc_type, ioc_value, threat_type, severity, source,
                       confidence, tags, first_seen, last_seen
                FROM threat_intel_iocs
                {where_clause}
                ORDER BY last_seen DESC
                LIMIT :limit OFFSET :offset
            """),
            params,
        )
        rows = result.fetchall()

    return {
        "iocs": [
            {
                "ioc_type": r[0], "ioc_value": r[1], "threat_type": r[2],
                "severity": r[3], "source": r[4], "confidence": r[5],
                "tags": r[6], "first_seen": str(r[7]), "last_seen": str(r[8]),
            }
            for r in rows
        ],
        "total": total,
        "limit": limit,
        "offset": offset,
    }
```

Verification:

#	Check	Expected
1	`POST /intel/sync`	`synced_pulses > 0`, `iocs_inserted > 0`
2	`GET /intel/iocs`	Non-empty list of IOCs
3	`GET /intel/iocs?ioc_type=ip`	Only IP-type IOCs
4	`SELECT COUNT(*) FROM threat_intel_iocs`	> 0 rows
5	Repeat sync → no duplicate errors	Upsert works (ON CONFLICT)
6	`GET /intel/feeds/status`	`otx_enabled: true`

TASK 3 — IOC Correlator: Add Domain Check (§11.3 Item 2) 🟤

Time Est: 45 min | **Priority:** 🟤 Critical
Source: MASTER_DOC_PART4 §11.3 — "Is the `dst_domain` (from DNS) in the IOC database? → If yes, flag as C2/phishing"

3.1 Add `check_domain()` to IOCCorrelator

```
async def check_domain(self, domain: str) -> Optional[Dict[str, Any]]:
    """
    Check if a domain exists in the threat_intel_iocs table.
    Per PART4 §11.3 item 2: flag as C2/phishing if matched.

    Args:
        domain: Domain name to look up (e.g., 'evil-c2.com').

    Returns:
        IOC data dict if found and active, None otherwise.
    """
    if not domain:
        return None

    async with async_session() as session:
        result = await session.execute(
            text(
                "SELECT ioc_value, threat_type, severity, source, "
                "       confidence, tags, first_seen, last_seen "
                "FROM threat_intel_iocs "
                "WHERE ioc_type = 'domain' "
                "  AND ioc_value = :domain "
                "  AND is_active = true "
                "ORDER BY confidence DESC "
                "LIMIT 1"
            ),
            {"domain": domain},
        )
        row = result.fetchone()
        if row:
            return {
                "ioc_value": row[0],
                "threat_type": row[1],
                "severity": row[2],
                "source": row[3],
                "confidence": row[4],
                "tags": row[5],
                "first_seen": str(row[6]) if row[6] else None,
                "last_seen": str(row[7]) if row[7] else None,
                "match_type": "c2_phishing",  # Per §11.3 item 2
            }
    return None
```

3.2 Update `correlate_flow()` to Include Domain Check

```
async def correlate_flow(self, flow_data: Dict[str, Any]) -> Dict[str, Any]:
    """
    Correlate a flow's IPs + domain against IOC database.
    Per §11.3: IP match → escalate, domain match → flag C2/phishing.
    """
    src_ip = flow_data.get("source_ip") or flow_data.get("src_ip")
    dst_ip = flow_data.get("dest_ip") or flow_data.get("dst_ip")
    dst_domain = flow_data.get("dst_domain") or flow_data.get("domain")

    src_match = await self.check_ip(src_ip) if src_ip else None
    dst_match = await self.check_ip(dst_ip) if dst_ip else None
    domain_match = await self.check_domain(dst_domain) if dst_domain else None

    has_match = any([src_match, dst_match, domain_match])

    escalation: Optional[str] = None
    flags: List[str] = []

    if has_match:
        # IP match → escalate severity
        if src_match or dst_match:
            match_severity = (src_match or dst_match or {}).get("severity",
"medium")
            escalation = "critical" if match_severity == "critical" else "high"

        # Domain match → flag as C2/phishing (§11.3 item 2)
        if domain_match:
            flags.append("c2_phishing")
            if not escalation:
                escalation = "high"

    return {
        "src_match": src_match,
        "dst_match": dst_match,
        "domain_match": domain_match,
        "has_ioc_match": has_match,
        "escalation_severity": escalation,
        "flags": flags,
    }
```

Verification:

#	Check	Expected
1	Insert test domain IOC	`INSERT INTO threat_intel_iocs (ioc_type, ioc_value, ...) VALUES ('domain', 'evil-c2.com', ...)`
2	`check_domain("evil-c2.com")`	Returns IOC data with `match_type: "c2_phishing"`

#	Check	Expected
3	`check_domain("google.com")`	Returns None
4	`correlate_flow({..., "dst_domain": "evil-c2.com"})`	`flags: ["c2_phishing"]`, `escalation: "high"`
5	`correlate_flow({..., "dst_domain": "google.com"})`	`flags: []`, no escalation

TASK 4 — VirusTotal Client + Hash Check (§11.3 Item 3) ⬛

Time Est: 60 min | **Priority:** ⬛ Critical
Source: MASTER_DOC_PART4 §11.3 — "Are any file hashes (if payload extracted) in VirusTotal? → If yes, flag as malware"

4.1 Create VirusTotal Client

Add to `backend/app/services/threat_intel.py`:

```
class VirusTotalClient:
    """VirusTotal file/IP/domain analysis client."""

    BASE_URL = "https://www.virustotal.com/api/v3"

    def __init__(self) -> None:
        self.api_key = os.environ.get("VIRUSTOTAL_API_KEY", "")
        self.enabled = bool(self.api_key)
        self._client: Optional[httpx.AsyncClient] = None
        if self.enabled:
            logger.info("[ThreatIntel] VirusTotal client initialized")
        else:
            logger.warning("[ThreatIntel] VIRUSTOTAL_API_KEY not set")

    async def _get_client(self) -> httpx.AsyncClient:
        if self._client is None or self._client.is_closed:
            self._client = httpx.AsyncClient(
                base_url=self.BASE_URL,
                headers={"x-apikey": self.api_key},
                timeout=30.0,
            )
        return self._client

    async def check_hash(self, file_hash: str) -> Dict[str, Any]:
        """
        Check a file hash against VirusTotal.
        Per §11.3 item 3: if detected → flag as malware.

        Args:
```

```
            file_hash: MD5, SHA1, or SHA256 hash.

        Returns:
            Dict with detection results.
        """
        if not self.enabled:
            return {"error": "VirusTotal not configured"}
        client = await self._get_client()
        try:
            resp = await client.get(f"/files/{file_hash}")
            resp.raise_for_status()
            data = resp.json().get("data", {}).get("attributes", {})
            stats = data.get("last_analysis_stats", {})
            return {
                "hash": file_hash,
                "malicious": stats.get("malicious", 0),
                "suspicious": stats.get("suspicious", 0),
                "undetected": stats.get("undetected", 0),
                "total_engines": sum(stats.values()) if stats else 0,
                "detection_ratio": f"{stats.get('malicious',
0)}/{sum(stats.values()) if stats else 0}",
                "is_malware": stats.get("malicious", 0) > 3,  # >3 engines flagged
                "file_type": data.get("type_description", "unknown"),
                "names": data.get("names", [])[:5],
            }
        except httpx.HTTPStatusError as e:
            if e.response.status_code == 404:
                return {"hash": file_hash, "is_malware": False, "message": "Not
found in VT"}
            logger.error("[VT] Hash check failed: %s", e)
            return {"hash": file_hash, "error": str(e)}
        except Exception as e:
            logger.error("[VT] Hash check failed: %s", e)
            return {"hash": file_hash, "error": str(e)}

    async def check_ip(self, ip: str) -> Dict[str, Any]:
        """Check IP reputation on VirusTotal."""
        if not self.enabled:
            return {"error": "VirusTotal not configured"}
        client = await self._get_client()
        try:
            resp = await client.get(f"/ip_addresses/{ip}")
            resp.raise_for_status()
            data = resp.json().get("data", {}).get("attributes", {})
            stats = data.get("last_analysis_stats", {})
            return {
                "ip": ip,
                "malicious": stats.get("malicious", 0),
                "suspicious": stats.get("suspicious", 0),
                "country": data.get("country", ""),
                "as_owner": data.get("as_owner", ""),
                "reputation": data.get("reputation", 0),
            }
        except Exception as e:
```

```
            logger.error("[VT] IP check failed: %s", e)
            return {"ip": ip, "error": str(e)}

    async def close(self) -> None:
        if self._client and not self._client.is_closed:
            await self._client.aclose()
```

4.2 Add `check_hash()` to IOCCorrelator

```
async def check_hash(self, file_hash: str) -> Optional[Dict[str, Any]]:
    """
    Check if a file hash exists in threat_intel_iocs table OR query VirusTotal.
    Per PART4 §11.3 item 3: if detected → flag as malware.
    """
    if not file_hash:
        return None

    # First, check local IOC database
    async with async_session() as session:
        result = await session.execute(
            text(
                "SELECT ioc_value, threat_type, severity, source, confidence "
                "FROM threat_intel_iocs "
                "WHERE ioc_type = 'hash' "
                "  AND ioc_value = :hash "
                "  AND is_active = true "
                "LIMIT 1"
            ),
            {"hash": file_hash},
        )
        row = result.fetchone()
        if row:
            return {
                "hash": row[0],
                "threat_type": row[1],
                "severity": row[2],
                "source": row[3],
                "confidence": row[4],
                "match_type": "malware",
            }

    # If not in local DB, try VirusTotal (on-demand per §11.2)
    try:
        from app.services.threat_intel import VirusTotalClient
        vt = VirusTotalClient()
        if vt.enabled:
            result = await vt.check_hash(file_hash)
            if result.get("is_malware"):
                return {
                    "hash": file_hash,
                    "threat_type": "malware",
```

```
                    "severity": "critical",
                    "source": "virustotal",
                    "confidence": 0.9,
                    "detection_ratio": result.get("detection_ratio", "0/0"),
                    "match_type": "malware",
                }
            await vt.close()
    except Exception as e:
        logger.error("[IOC] VT hash check failed: %s", e)

    return None
```

4.3 Update `correlate_flow()` to Include Hash Check

Add to the existing `correlate_flow()` method:

```
# File hash check (if payload extracted) — §11.3 item 3
file_hash = flow_data.get("file_hash") or flow_data.get("payload_hash")
hash_match = await self.check_hash(file_hash) if file_hash else None

# ... in the has_match check:
has_match = any([src_match, dst_match, domain_match, hash_match])

# ... in the flags section:
if hash_match:
    flags.append("malware")
    escalation = "critical"  # Malware → always CRITICAL
```

4.4 Update ThreatIntelService

```
class ThreatIntelService:
    def __init__(self) -> None:
        self.otx = OTXClient()
        self.abuseipdb = AbuseIPDBClient()
        self.virustotal = VirusTotalClient()  # ADD THIS
        self.stats = {"lookups": 0, "iocs_found": 0}

    def get_status(self) -> Dict[str, Any]:
        return {
            "otx_enabled": self.otx.enabled,
            "abuseipdb_enabled": self.abuseipdb.enabled,
            "virustotal_enabled": self.virustotal.enabled,  # ADD THIS
            "stats": self.stats,
        }

    async def close(self) -> None:
        await self.otx.close()
```

```
        await self.abuseipdb.close()
        await self.virustotal.close()  # ADD THIS
```

Verification:

#	Check	Expected
1	VT client initialized	`[ThreatIntel] VirusTotal client initialized` in logs
2	`GET /intel/feeds/status`	`virustotal_enabled: true`
3	`check_hash("known_malware_hash")` via VT API	Returns detection ratio
4	`check_hash("abcdef...")` (unknown)	Returns None or "Not found in VT"
5	`correlate_flow({..., "file_hash": "malware_hash"})`	`flags: ["malware"]`, `escalation: "critical"`
6	Insert hash IOC to local table	`check_hash()` finds it locally first

TASK 5 — Verify Live IOC Correlation 🔴

Time Est: 30 min | **Priority:** 🔴 Critical
Source: Integration verification

After Tasks 1-4 complete, run the full correlation chain:

5.1 Sync OTX Feed

```
curl -s -X POST http://localhost:8000/api/v1/intel/sync | python3 -m json.tool
# Expected: synced_pulses > 0, iocs_inserted > 0
```

5.2 Verify IOC Table Populated

```
SELECT ioc_type, COUNT(*) FROM threat_intel_iocs WHERE is_active = true GROUP BY
ioc_type;
```

Expected:

```
 ioc_type | count
----------+-------
 ip       |   50+
 domain   |   30+
 hash     |   20+
```

5.3 Test IP Lookup Against Populated Table

```
# Pick a known malicious IP from OTX pulses
curl -s http://localhost:8000/api/v1/intel/lookup/X.X.X.X | python3 -m json.tool
```

5.4 Test IOC Correlation

```
# Run in docker compose exec backend python
from app.services.ioc_correlator import IOCCorrelator
import asyncio

async def test():
    c = IOCCorrelator()
    # Use a real IOC IP from the synced table
    result = await c.correlate_flow({"src_ip": "1.2.3.4", "dst_ip":
"MALICIOUS_IP_FROM_TABLE"})
    print(result)

asyncio.run(test())
```

Verification:

#	Check	Expected
1	OTX sync populates IOC table	> 0 IPs, domains, hashes
2	AbuseIPDB IP lookup works	Returns abuse_confidence
3	IOCCorrelator matches a real OTX IOC	`has_ioc_match: True`
4	Alert auto-escalated on IOC match	`severity` upgraded to HIGH or CRITICAL
5	`GET /intel/iocs` returns real data	Non-empty list
6	`GET /intel/feeds/status`	All 3 providers enabled

TASK 6 — Apply Tuned IF Parameters 🟢

Time Est: 30 min | **Priority:** 🟢 Medium
Source: Day 13 Tuning Results (best_params.json)

Retrain Isolation Forest with the optimized parameters from hyperparameter tuning.

6.1 Apply via /ml/retrain

```
curl -s -X POST http://localhost:8000/api/v1/ml/retrain \
  -H 'Content-Type: application/json' \
  -d '{"dataset": "nsl_kdd", "models": ["isolation_forest"]}' | python3 -m
json.tool
```

6.2 Alternative: Direct Retrain with best_params

If the retrain endpoint doesn't yet read from best_params.json, manually update `ml/training/hyperparams.py` with the tuned values:

```
# isolation_forest params:
{
    "n_estimators": 100,     # verified via tuning (was 100, stays 100)
    "contamination": 0.10,   # CHANGED from 0.05 → 0.10
    "max_samples": 1024,     # CHANGED from 256 → 1024
}
```

Then run:

```
docker compose exec ml-worker python -m ml.training.train_all --dataset nsl_kdd
```

6.3 Verify New Model Performance

```
docker compose exec ml-worker python -m ml.training.evaluate
```

Expected improvement:

- IF Accuracy: 79.68% → 82.54% (+2.86%)
- IF F1: 78.75% → 83.03% (+4.28%)

⚠ **CRITICAL: Ensemble weights (0.30/0.45/0.25) and thresholds remain LOCKED.**

Verification:

#	Check	Expected
1	IF retrained with tuned params	New model saved
2	IF accuracy ≥ 82%	Up from 79.68%
3	IF F1 ≥ 83%	Up from 78.75%
4	Ensemble weights unchanged	0.30/0.45/0.25
5	Alert thresholds unchanged	0.90/0.75/0.50/0.30

#	Check	Expected
6	ML Worker loads new model	Log: "[ModelManager] Loaded isolation_forest"
7	Live scoring still works	New flows scored with tuned model

TASK 7 — PCAP Upload Pipeline (Scaffold) ⚪

Time Est: 45 min | **Priority:** ⚪ Medium
Source: MASTER_DOC_PART2 §5.1 — `POST /capture/upload-pcap`

7.1 Add Endpoint to capture.py

```
from fastapi import UploadFile, File

@router.post("/upload-pcap")
async def upload_pcap(file: UploadFile = File(...)):
    """
    Upload a PCAP file for historical analysis.
    Per MASTER_DOC_PART2 §5.1.

    The capture engine's pcap_processor.py handles parsing.
    Flows extracted from PCAP are scored by the ML ensemble.
    """
    if not file.filename.endswith(('.pcap', '.pcapng', '.cap')):
        raise HTTPException(status_code=400, detail="Only .pcap/.pcapng/.cap files
accepted")

    # Save to temp file
    import tempfile, os
    with tempfile.NamedTemporaryFile(delete=False, suffix=".pcap") as tmp:
        content = await file.read()
        tmp.write(content)
        tmp_path = tmp.name

    # Process PCAP in background
    task_id = str(uuid.uuid4())[:8]
    asyncio.create_task(_process_pcap(task_id, tmp_path, file.filename))

    return {
        "status": "processing",
        "task_id": task_id,
        "filename": file.filename,
        "size_bytes": len(content),
    }
```

Verification:

#	Check	Expected

#	Check	Expected
1	POST /capture/upload-pcap with .pcap	200, task_id returned
2	POST with non-.pcap file	400 error
3	OpenAPI docs show endpoint	Visible
4	API coverage	38/42 (90.5%)

Post-Task API Coverage Update

Service	Before	After	Coverage
Auth	5/5	5/5	**100%**
Flows	6/6	6/6	**100%**
Alerts	5/5	5/5	**100%**
Capture	4/5	**5/5**	**100%** ☑ (+upload-pcap)
System	2/3	2/3	67%
WebSocket	1/1	1/1	**100%**
ML	5/5	5/5	**100%**
LLM	5/5	5/5	**100%**
Intel	4/4	4/4	**100%**
Reports	0/3	0/3	Week 6
TOTAL	**37/42**	**38/42**	**90.5%**

Files Modified / Created (Expected)

File	Action	Lines (est.)
`app/services/ioc_correlator.py`	MODIFY	+60 (domain + hash checks)
`app/services/threat_intel.py`	MODIFY	+80 (VirusTotalClient class)
`app/api/v1/intel.py`	MODIFY	+50 (sync + list_iocs)
`app/api/v1/capture.py`	MODIFY	+30 (upload-pcap)
`ml/training/hyperparams.py`	MODIFY	+3 (tuned IF params)

Risk Assessment

Risk	Impact	Mitigation
OTX rate limiting	Sync returns 0 pulses	Retry with smaller limit

Risk	Impact	Mitigation
AbuseIPDB daily cap (1K)	Too many lookups	Cache results in threat_intel_iocs
VT daily cap (500)	Hash checks throttled	Local IOC table as primary, VT as fallback
OTX empty pulses	No IOCs to insert	Subscribe to popular OTX pulse authors
Tuned IF diverges with live data	Worse detection	Monitor for 24h before committing

---

STRICT RULES REMINDER

1. **DO NOT** change ensemble weights (0.30/0.45/0.25) — LOCKED
2. **DO NOT** change alert thresholds (0.90/0.75/0.50/0.30) — LOCKED
3. **DO NOT** suggest Kafka, Kubernetes, Elasticsearch, MongoDB
4. **DO NOT** add features not in the 10 modules
5. **DO NOT** use Tailwind CSS — Vanilla CSS + CSS Variables only
6. LLM via **OpenRouter only** — 3 verified models
7. Prompts follow **PART4 §9.2** templates
8. Master documentation (5 parts) is the **SOLE source of truth**
9. All code: **typed, error-handled, documented, production-quality**
10. Python: **type hints, async/await, SQLAlchemy 2.x**

Day 14 Worklog — Week 4 Day 1
E2E Pipeline LIVE: capture → ML (24,700+ flows) → alerts → IOC → LLM narrative → WebSocket
Target: §11.3 full compliance (IP + domain + hash) + API keys + tuned IF params + PCAP upload
Ensemble: 80.73% acc | 80.96% F1 | 0.9312 AUC-ROC (LOCKED)
Post-Day 14 target: 38/42 API coverage (90.5%)